{EPITECH}

BOOTSTRAP WOLF3D

DISCOVER “3D WORLD”



BOOTSTRAP WOLF3D

The goal of this project is to understand and implement a simple 3D simulation from a 2D world, using the raycasting technique. The rendering is done with CSFML.

This document guides you step by step, with the involved files, functions to implement, practical tips, common mistakes to avoid, etc.



Step 1 — Initializing the Engine



In this first step, we prepare the engine's foundations: creating the window, drawing the floor/ceiling, initializing the player, and detecting walls.

Functions to Implement

```
void init_player(Player *player); // Initializes the player's position and angle
int is_wall(int x, int y);        // Returns 1 if the position corresponds to a wall
void draw_floor_and_ceiling(sfRenderWindow *window); // Draws the floor and ceiling of the
    screen
```

Data structures

```
typedef struct {
    float x;
    float y;
    float angle;
} Player;

#define TILE_SIZE 64
#define MAP_WIDTH 8
#define MAP_HEIGHT 8

extern int map[MAP_HEIGHT][MAP_WIDTH];
```

Objectives

- ✓ Create a CSFML 800x600 window
- ✓ Represent the 2D map with an array
- ✓ Initialize the player with a position and a view angle
- ✓ Implement a function is_wall to detect obstacles



- ✓ To convert from **pixel coordinates** to **tile coordinates**: coord / TILE_SIZE
- ✓ To **normalize an angle**: fmod(angle, 2 * M_PI)

{EPITECH}

Step 2 — Raycasting Implementation



This is where pseudo-3D truly begins: we send rays in front of the player and measure the distance to the walls.

Functions to implement

```
void cast_all_rays(sfRenderWindow *window, Player player);
float cast_single_ray(Player player, float ray_angle, sfRenderWindow *window);
```

Usefull constants

```
#define FOV (M_PI / 3)
#define NUM_RAYS 800
```

Objectives

- ✓ Sweep the field of view with **rays**
- ✓ Detect the first **collision with a wall** for each ray
- ✓ Measure the **distance** from the player to the wall



- ✓ Correcting the **fish-eye effect**: multiply the distance by cos(player.angle - ray_angle)
- ✓ Use a simple step (0.1) for ray progression..



If your walls seem to ripple or display irregularly, it's probably a non-normalized angle issue!

{EPITECH}

Step 3 — 3D Wall Projection



This step transforms the ray distances into vertical columns on the screen. The closer a wall is, the larger it appears!

Functions to implement

```c
void render_wall_column(sfRenderWindow *window, int column, float wall_height, sfColor
    color);
```

Objectives

- ✓ Display a **vertical column** for each ray
- ✓ Use the inverse of the distance for **height calculation**
- ✓ Vary the **color** based on the wall's orientation for better readability

Need help with maths ? :p

```c
float corrected_dist = raw_dist * cos(ray_angle - player.angle);
float wall_height = (TILE_SIZE / corrected_dist) * (WIDTH / 2);
```



Try varying the hue (e.g., darker for the north face, lighter for the east) to enhance the depth effect!

Step 4 — Player Movement



A rendering engine without movement would be quite dull. Let's bring our player to life!

Functions to implement

```c
void update_player(Player *player);
int is_wall(int x, int y);
```

Objectives

- ✓ Movement via keyboard:
  - Z / S: move forward / backward
  - Q / D: turn left / right
- ✓ Prevent movement through walls



Add a rotation speed (rotation_speed) and movement speed (move_speed) to adjust the fluidity of the movement.

Happy exploring in the **3D world from 2D**!

v 1.1

